Instituto Tecnológico De Tijuana

Departamento De Sistemas Y Computación

Ingeniería En Sistemas Computacionales

Analisi avanzados de software

Unidad:

Unidad 2

Tema:

Actividad de evidencia 2_U2 Singleton

Integrantes:

Garcia Calderon Angel Fernando - 22210306

Docente:

MARIBEL GUERRERO LUIS

Tijuana, B.C. 7 de Marzo Del 2026

Singleton

Identificar y explicar:

1.-el atributo estático de la instancia

```
public class Central_911
{
    private static Central_911 _instance;
    private static readonly object _lock = new object();
```

El private static Central_911 _instance, es el atributo estático de la instancia, porque es el único que se encuentra en todo el código y se encarga de almacenar la única instancia que se encuentra.

2.-el constructor

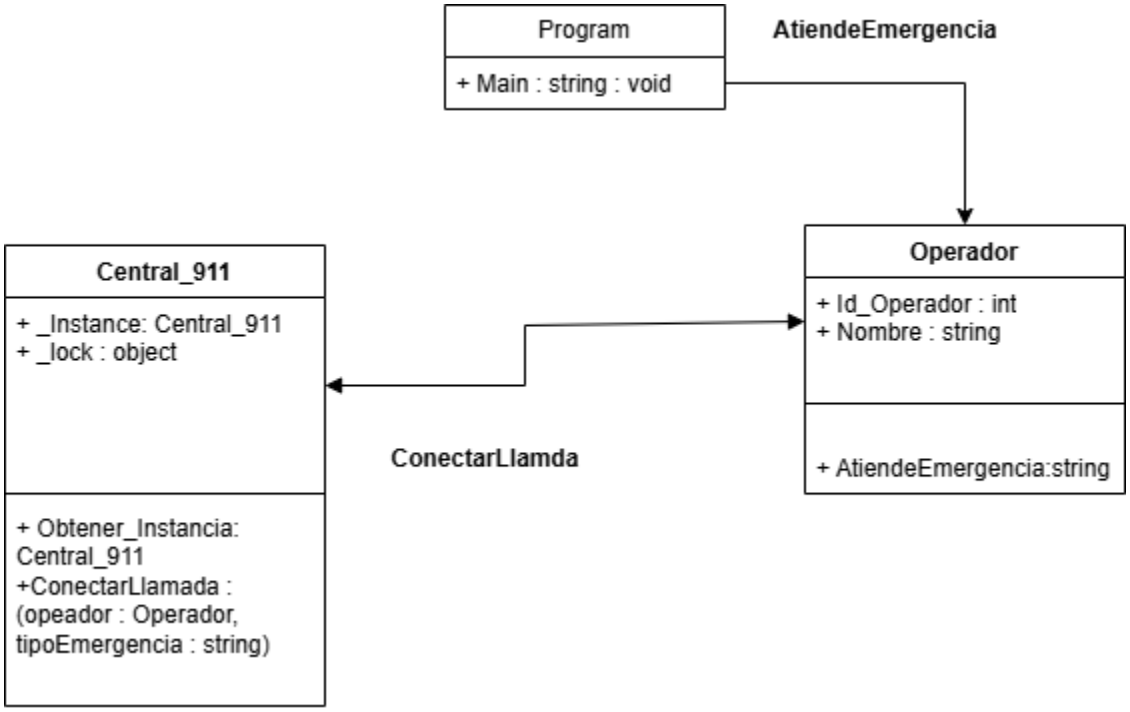
```
private Central_911()
{
    Central = "Central 911";
}
```

El constructor es privado, por lo cual no se le podría crear objetos desde otra clase.

¿Por qué no se pueden crear múltiples objetos?

Porque el singleton lo que hace es impedir que mediante nuestro constructor y instancia única, por que no se puede usar new y solo se guarda la instancia en global, que es única.

Diagrama UML



Agregados

Se agregaron mas llamadas y mas operadores. Verificando una sola instancia. Porque todas todas las llamadas apuntan a la misma instancia de la clase Central_911.

```
internal class Program
{
    static void Main(string[] args)
    {
        Central_911 llamada1 = Central_911.Obtener_Instancia();
        Central_911 llamada2 = Central_911.Obtener_Instancia();
        Central_911 llamada3 = Central_911.Obtener_Instancia();
        Central_911 llamada4 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");
        Operador op3 = new Operador(3, "Miguel");
        Operador op4 = new Operador(4, "Ana Gastelum");
        Operador op5 = new Operador(5, "Pedro");
        Operador op6 = new Operador(6, "Moises");

        llamada1.ConectarLlamada(op1, "Incendio");
        llamada2.ConectarLlamada(op2, "Violeta");
        llamada3.ConectarLlamada(op3, "Accidente");
        llamada4.ConectarLlamada(op4, "Intento de suicidio");

        llamada1.ConectarLlamada(op5, "Robo");
        llamada2.ConectarLlamada(op6, "Emergencia medica");
        llamada3.ConectarLlamada(op2, "Violencia domestica");
        llamada4.ConectarLlamada(op1, "Accidente");

        Console.WriteLine("\nVerificando instancia única:");

        Console.WriteLine("llamada1 == llamada2: " + ReferenceEquals(llamada1, llamada2));
        Console.WriteLine("llamada1 == llamada3: " + ReferenceEquals(llamada1, llamada3));
        Console.WriteLine("llamada1 == llamada4: " + ReferenceEquals(llamada1, llamada4));

        Console.ReadKey();
    }
}
```

Más casos de llamada.

```
            break;

        case "Robo":
            Console.WriteLine("Enviando patrullas al lugar");
            break;

        case "Emergencia medica":
            Console.WriteLine("Enviando ambulancia");
            break;

        case "Violencia domestica":
            Console.WriteLine("Enviando patrulla especializada");
            break;

        default:
            Console.WriteLine("Tipo de emergencia no reconocido.");
            break;
    }
```

```
 Considere la posibilidad de agregar el modificador "required'"o declarar el campo como un valor que acepta valores NULL.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla

Llamada conectada con el operador Miguel
Operador Miguel atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales

Llamada conectada con el operador Ana Gastelum
Operador Ana Gastelum atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Pedro
Operador Pedro atendiendo emergencia de tipo: Robo
Enviando patrullas al lugar

Llamada conectada con el operador Moises
Operador Moises atendiendo emergencia de tipo: Emergencia medica
Enviando ambulancia

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violencia domestica
Enviando patrulla especializada

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales

Verificando instancia única:
llamada1 == llamada2: True
llamada1 == llamada3: True
llamada1 == llamada4: True
```

Conclusión

El patrón singleton funciona con una sola instancia que en este caso es la central 911, donde llegan todas las llamadas y se les asigna un operador que atenderá la emergencia. El constructor nos ayuda a evitar que se creen más objetos desde otra clase, y el método público que ayuda a controlar y el acceso a la instancia.